



ค่ายโอลิมปิกวิชาการ วิชาคอมพิวเตอร์

อัลกอริทึมการย้อนรอย (Backtracking algorithm)

สรวิ รัตนสัญญา

san.r@sci.kmutnb.ac.th

ภาควิชาวิทยาการคอมพิวเตอร์และสารสนเทศ

มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ



หัวข้อ

- ปัญหาเชิงการจัดและการสลับที่ (Combinatorial and Permutation problems)
- การค้นปริภูมิสถานะ (State space searching)
- การตะลุยหาคำตอบ (Brute-force)
- อัลกอริทึมการย้อนรอย (Backtracking algorithm)
- ตัวอย่าง
- แบบฝึกหัด

ปัญหาเชิงการจัดและการสลับที่

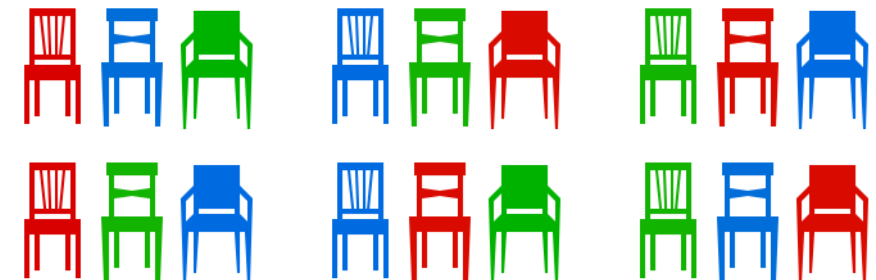
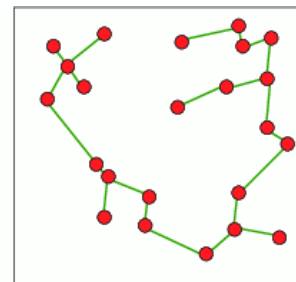
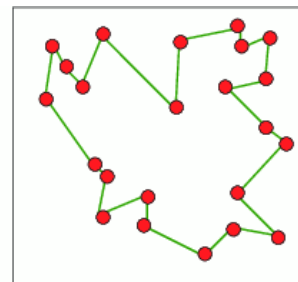
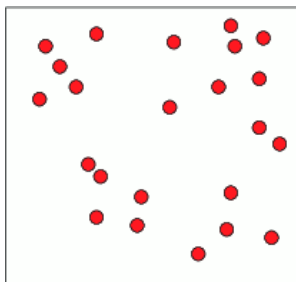
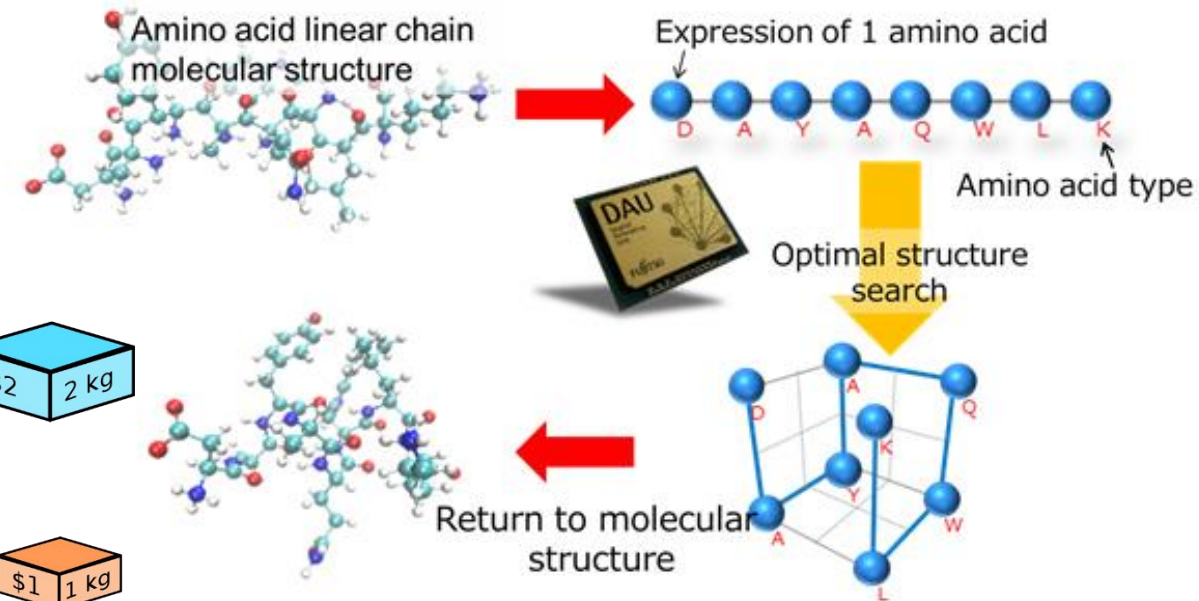
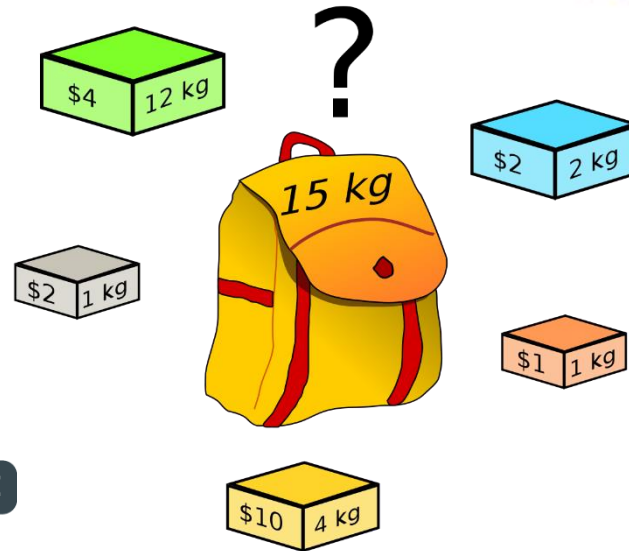
(Combinatorial and Permutation problems)

- พบได้บ่อยในชีวิตประจำวัน
- ลักษณะของปัญหาเชิงการจัด
 - คำตอบของปัญหาอยู่ในรูปเซตของทางเลือก
 - เป็นเซตแบบมีลำดับ หรือไม่มีลำดับ
 - มีคำตอบที่เป็นไปได้มากกว่าหนึ่งคำตอบ ขึ้นกับเงื่อนไขของปัญหานั้น ๆ
 - เลือกคำตอบที่เหมาะสมที่สุด (Optimal solution)



ปัญหาเชิงการจัดและการสลับที่ (Combinatorial and Permutation problems)

■ ตัวอย่างของปัญหา



ทบทวนการเรียงสับเปลี่ยนและการจัดกลุ่ม

	With repetition	Without repetition
❖ Permutation: Order is important.	n^r	$n_{Pr} = \frac{n!}{(n-r)!}$
MS Excel Formula >>	PERMUTATIONA(n,r)	PERMUT(n,r)
❖ Combination: Order is NOT important.	$\frac{(r+n-1)!}{r!(n-1)!}$	$n_{Cr} = \frac{n!}{(n-r)!r!}$
MS Excel Formula >>	COMBINA(n,r)	COMBIN(n,r)

QG

$$\binom{n+k-1}{k} = \text{Stars and Bars} = \binom{n+k-1}{n-1}$$



Stars and Bars



PERMUTATIONS / COMBINATIONS SUMMARY

Permutation

```
29 int main() {
30     int n = 3, i;
31     int x[n+1];
32
33     for(i=1; i<=n; i++)
34     { x[i] = i;
35     }
36     permute(x, 1, n);
37     return 0;
38 }
1  #include <stdio.h>
2  void print(int x[], int n)
3  {
4      for(int i=1; i<=n; i++)
5      { printf("%d ", x[i]);
6      }
7      printf("\n");
8  }
```

```
17 void permute(int x[], int l, int r)
18 { if ( l == r)
19     print(x, r);
20 else {
21     for(int i = l; i<=r ; i++)
22     { swap(&x[l], &x[i]);
23       permute(x, l+1, r);
24       swap(&x[l], &x[i]);
25     }
26 }
27 }
```

```
10 void swap(int *x, int *y)
11 { int tmp = *x;
12   *x = *y;
13   *y = tmp;
14 }
```

Subset

```

24 int main()
25 {   int n = 3;
26     int x[n + 1];
27     Subset1(x, 0, n);
28     return 0;
29 }

```

```

1  #include <stdio.h>
2
3  int A[] = {6, 7, 8};
4

```

```

13 void Subset1(int x[], int l, int r)
14 {   if (l == r)
15     print_sol(x, r);
16     else
17     {   x[l+1] = 1;
18         Subset1(x, l+1, r);
19         x[l+1] = 0;
20         Subset1(x, l+1, r);
21     }
22 }

```

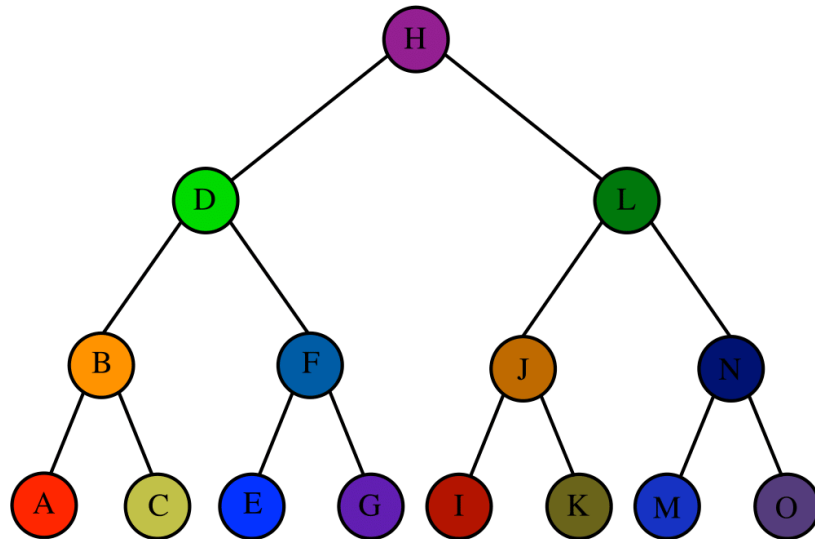
```

5 void print_sol(int x[], int n)
6 {   for (int i = 1; i <= n; i++)
7     {   if (x[i]== 1)
8         printf("%d ", A[i-1]);
9     }
10    printf("\n");
11 }

```

การค้นปริภูมิสถานะ (State space searching)

- ปริภูมิสถานะ คือ คำตอบทั้งหมดที่เป็นไปได้ทั้งหมดของปัญหานั้น ๆ
- เราสามารถหาคำตอบที่เหมาะสมจากการค้นหาในปริภูมิสถานะ



G?

- Permutation
- Subset
- Set partition

ทบทวนการท่องต้นไม้

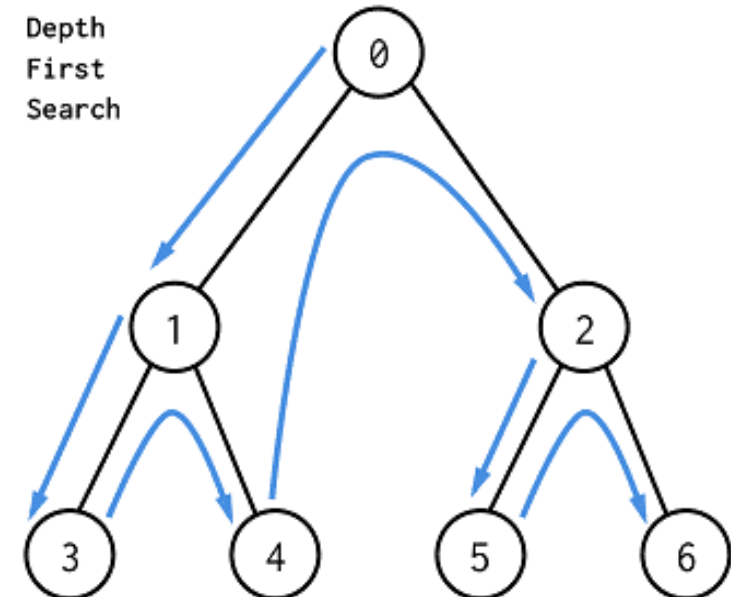
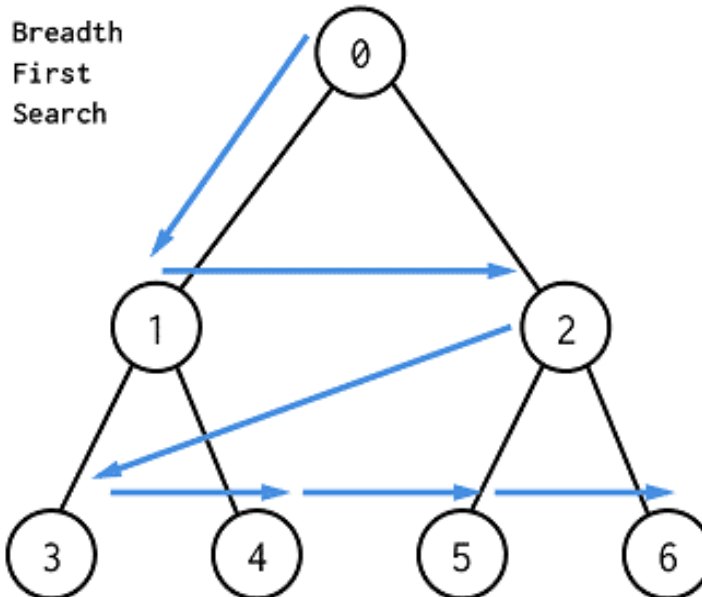
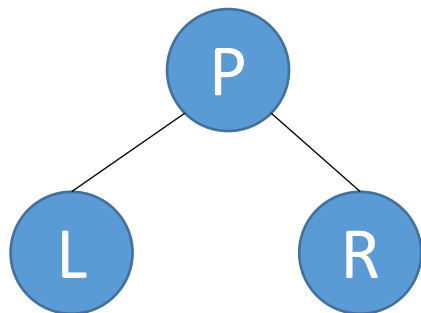
■ Breadth first search – Level-order

■ Depth first search

■ Pre-order (PLR)

■ In-order (LPR)

■ Post-order (LRP)



DFS vs BFS

```
1 #include<iostream>
2 #include<vector>
3 #include<queue>
4 #include<stack>
5 using namespace std;
6 //add the edge in graph
7 void edge(vector<int>adj[],int u,int v){
8     adj[u].push_back(v);
9 }
```

```
11 void bfs(int s,vector<int>adj[],bool visit[]){
12     queue<int>q;//queue in STL
13     q.push(s);
14     visit[s]=true;
15     while(!q.empty()){
16         int u=q.front();
17         cout<<u<<" ";
18         q.pop();
19         //loop for traverse
20         for(int i=0;i<adj[u].size();i++){
21             if(!visit[adj[u][i]]){
22                 q.push(adj[u][i]);
23                 visit[adj[u][i]]=true;
24             }
25         }
26     }
27 }
```

```
29 void dfs(int s, vector<int>adj[], bool visit[]){
30     stack<int>stk; //stack in STL
31     stk.push(s);
32     visit[s]=true;
33     while(!stk.empty()){
34         int u=stk.top();
35         cout<<u<<" ";
36         stk.pop();
37         //loop for traverse
38         for(int i=0; i<adj[u].size(); i++){
39             if(!visit[adj[u][i]]){
40                 stk.push(adj[u][i]);
41                 visit[adj[u][i]]=true;
42             }
43         }
44     }
45 }
```

```
47 int main(){
48     vector<int>adj[5]; //vector of array to store the graph
49     bool visit[5]; //array to check visit or not of a node
50     //initially all node are unvisited
51     for(int i=0; i<5; i++){
52         visit[i]=false;
53     }
54     //input for edges
55     edge(adj, 0, 2);
56     edge(adj, 0, 1);
57     edge(adj, 1, 3);
58     edge(adj, 2, 0);
59     edge(adj, 2, 3);
60     edge(adj, 2, 4);
```

```
61 cout<<"BFS traversal is"<<" ";
62 //call bfs funtion
63 bfs(0,adj,visit);//1 is a starting point
64 cout<<endl;
65 //again initialise all node unvisited for dfs
66 for(int i=0;i<5;i++){
67     visit[i]=false;
68 }
69 cout<<"DFS traversal is"<<" ";
70 //call dfs function
71 dfs(0,adj,visit);//1 is a starting point
72 }
```

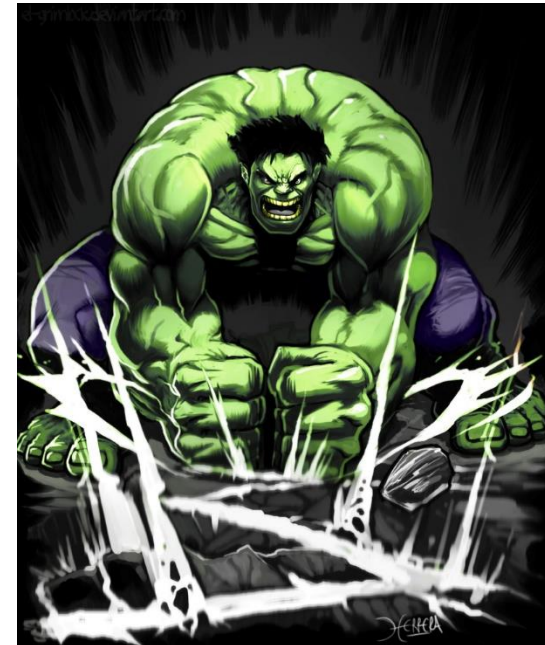


การค้นปริภูมิสถานะ (State space searching)

- เราต้องทำการสร้างปริภูมิสถานะเพื่อนำมาใช้ค้นหาคำตอบ ซึ่งสามารถทำได้ 2 วิธี
 - สร้างแบบไม่มีแบบแผน (Non-systematic)
 - สร้างแบบมีแบบแผน (Systematic) – การเรียงสับเปลี่ยน และการจัดหมู่
- การค้นปริภูมิสถานะ สามารถทำได้ 2 วิธี
 - การค้นหาจนหมดแรง (Exhaustive searching) หรือ การตะลุยคำตอบ (Brute-force)
 - การย้อนรอย (Backtracking)

การตะลุยคำตอบ (Brute-force)

- เป็นการแก้ปัญหาย่างตรงไปตรงมา เข้าใจง่าย
- โดยทำการตรวจสอบทุกเซตคำตอบเพื่อหาเซตคำตอบที่เหมาะสมที่สุด ตามเงื่อนไขของปัญหา (Constraint Satisfaction)
- เหมาะกับปัญหขนาดเล็กที่มีจำนวน input ไม่มาก
- ตัวอย่างเช่น
 - Bubble sort, Selection sort, Insertion sort
 - Sequential search
 - DFS และ BFS



อัลกอริทึมการย้อนรอย (Backtracking algorithm)

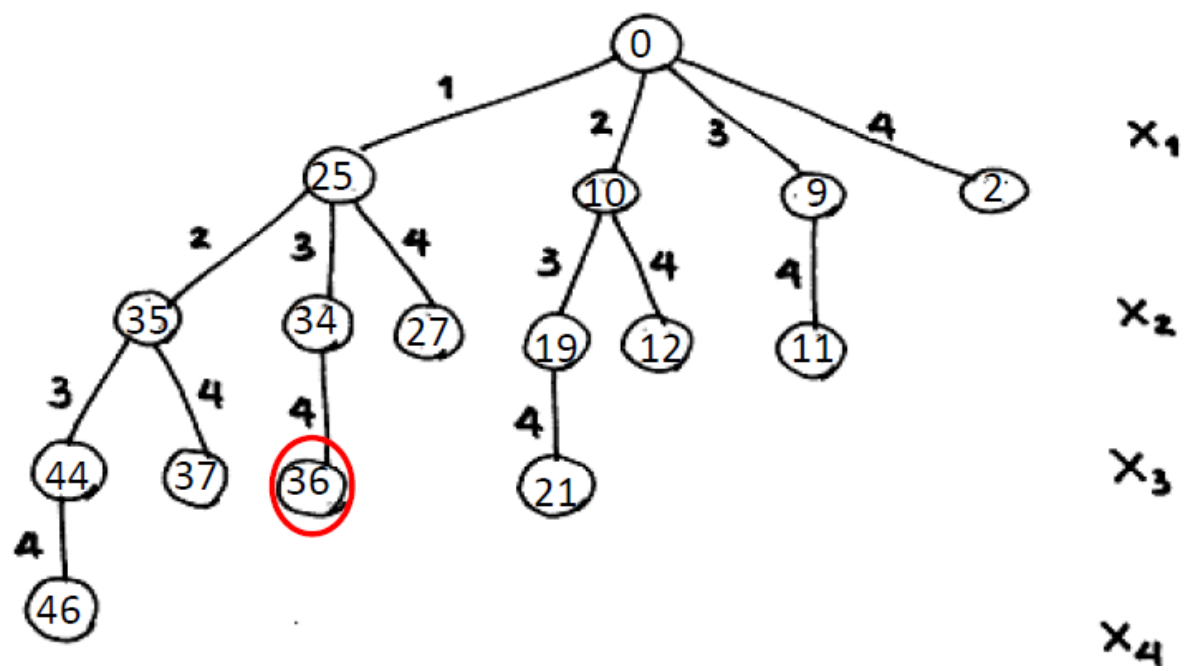
- ทำการตรวจสอบเฉพาะคำตอบที่อยู่ภายใต้เงื่อนไขของปัญหา
- โดยการสร้างชุดคำตอบทีละส่วน และพิจารณาว่าอยู่ภายใต้เงื่อนไขของปัญหาหรือไม่
- หากยังอยู่ภายใต้เงื่อนไข จะทำการสร้างส่วนของคำตอบถัดไป และทำการตรวจสอบใหม่
- หากไม่ จะทำการย้อนกลับ ไปยังชุดคำตอบก่อนหน้านี้ และสร้างชุดคำตอบใหม่



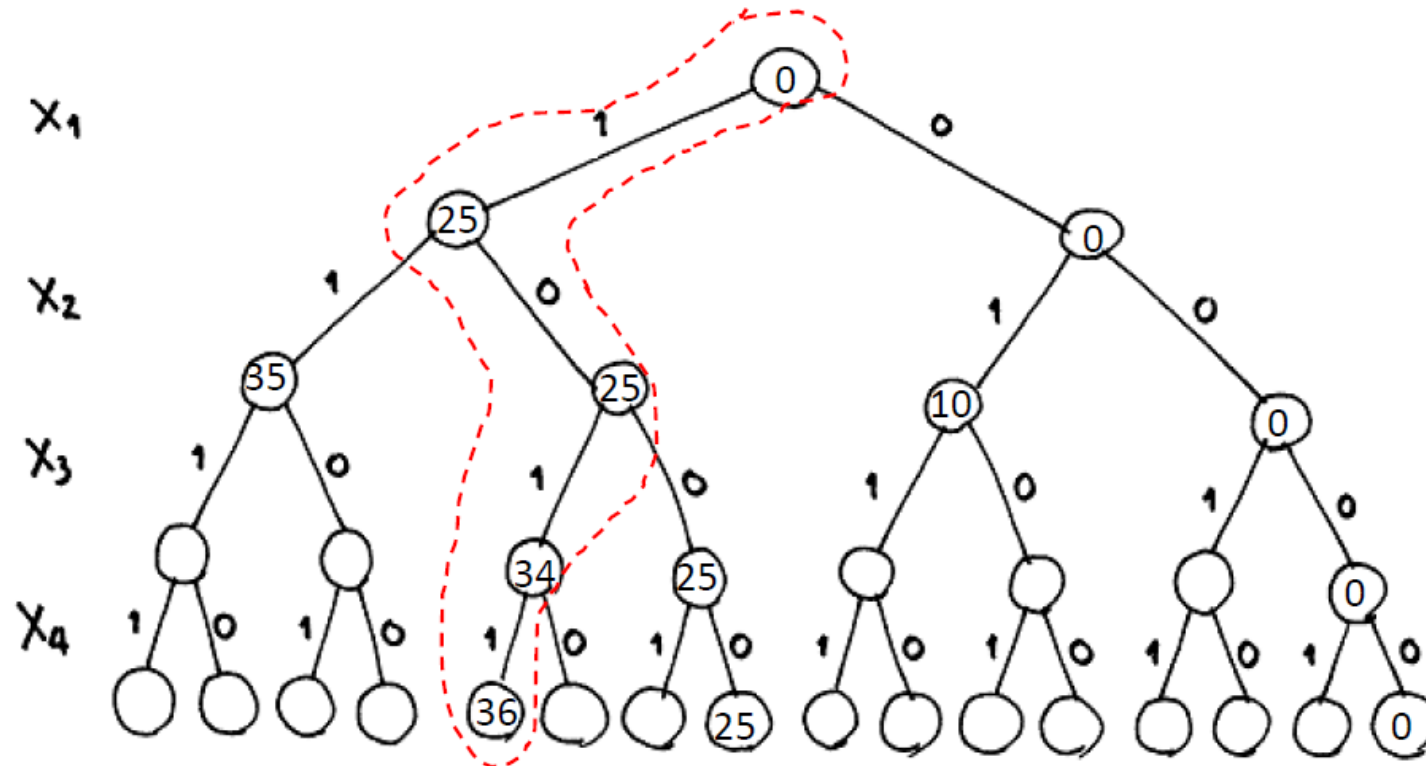
- ปริภูมิสถานะ คือต้นไม้ (tree) ที่แสดงคำตอบที่เป็นไปได้ทั้งหมด โดยที่
 - โหนด (node) แทน สถานะของคำตอบ และ
 - เส้นเชื่อม (edge) แทน วิธีการที่ได้มาซึ่งคำตอบ
- ในการสร้างปริภูมิสถานะ สามารถทำได้ 2 แบบ
 - แบบที่ 1 แสดงคำตอบโดยเก็บลำดับของสมาชิกในเซตที่ถูกเลือกเป็นคำตอบ
 - แบบที่ 2 แสดงคำตอบโดยเก็บสถานะของสมาชิกในเซตที่ถูกเลือกเป็นคำตอบ

ให้ $A = \{25, 10, 9, 2\}$ จงหาเซตย่อยที่ผลรวมของสมาชิกมีค่าเท่ากับ 36

แบบที่ 1 แสดงคำตอบโดยเก็บลำดับของสมาชิกในเซต $A \{x_1, x_2, \dots, x_k\}$ ซึ่งถูกเลือกเป็นคำตอบ โดยที่ $1 \leq k \leq n$



แบบที่ 2 แสดงคำตอบโดยเก็บสถานะของแต่ละสมาชิกใน A ซึ่งถูกเลือก $\{x_1, x_2, \dots, x_n\}$ โดยที่ $x_i \in \{0, 1\}$ หรือไบนารีเซต (binary set)



Backtracking Example

```

1  #include <stdio.h>
2  #define MAX_N 100
3  int n, s, a[MAX_N], x[MAX_N];
4  void print_solution() {
5      int i;
6      for (i = 0; i < n; i++) {
7          if (x[i]) {
8              printf("%d ", a[i]);
9          }
10     }
11     printf("\n");
12 }

```

```

13 void backtrack(int i, int sum) {
14     int j;
15     if (i == n) {
16         if (sum == s)
17             print_solution();
18     }
19     else {
20         for (j = 0; j <= 1; j++) {
21             x[i] = j;
22             backtrack(i + 1, sum + j * a[i]);
23         }
24     }
25 }
26 }

```

```

27 int main() {
28     a[0] = 25; a[1] = 10; a[2] = 9; a[3] = 2;
29     n = 4;
30     s = 12;
31     backtrack(0, 0);
32     return 0;
33 }

```

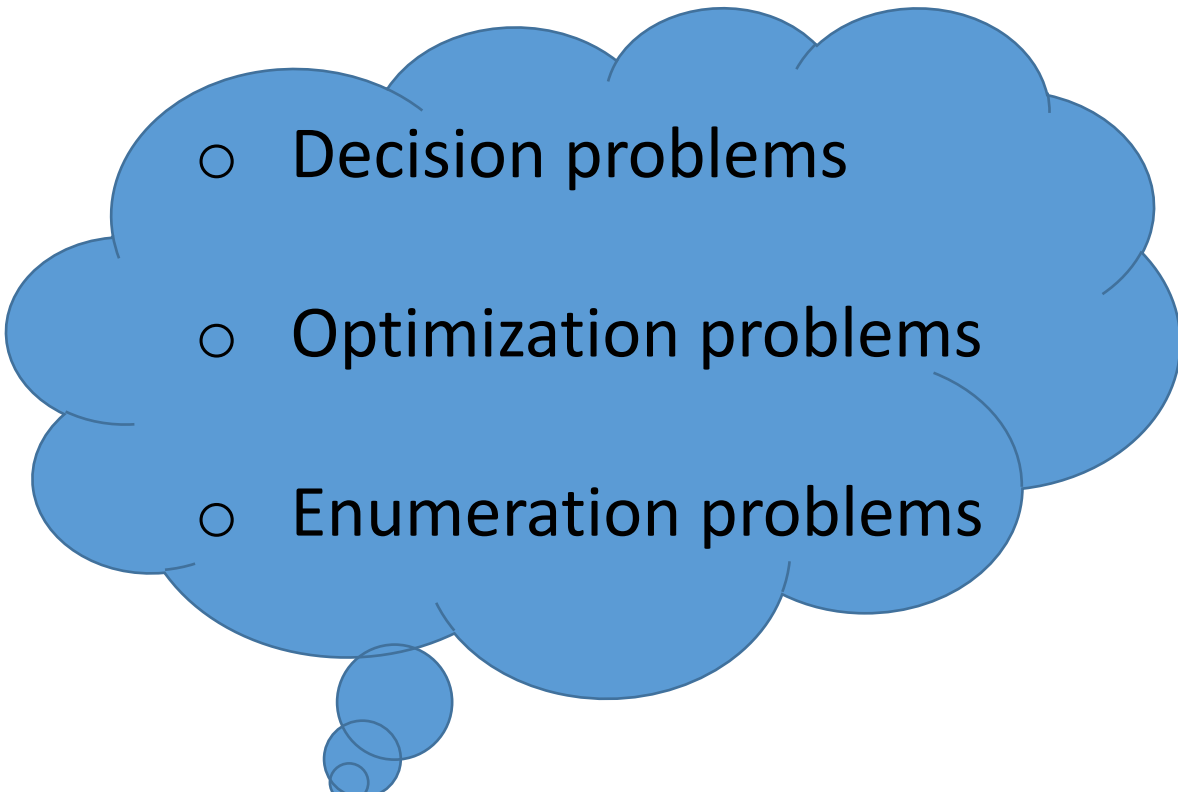
Backtracking Example

- Bounding function

```
void backtrack(int i, int sum) {  
    int j;  
    if (i == n) {  
        if (sum == s)  
            print_solution();  
    }  
    else { if(sum+a[i+1] <= s)           // bounding function  
        { for (j = 0; j <= 1; j++)  
            { x[i] = j;  
              backtrack(i + 1, sum + j * a[i]);  
            }  
        }  
    }  
}
```

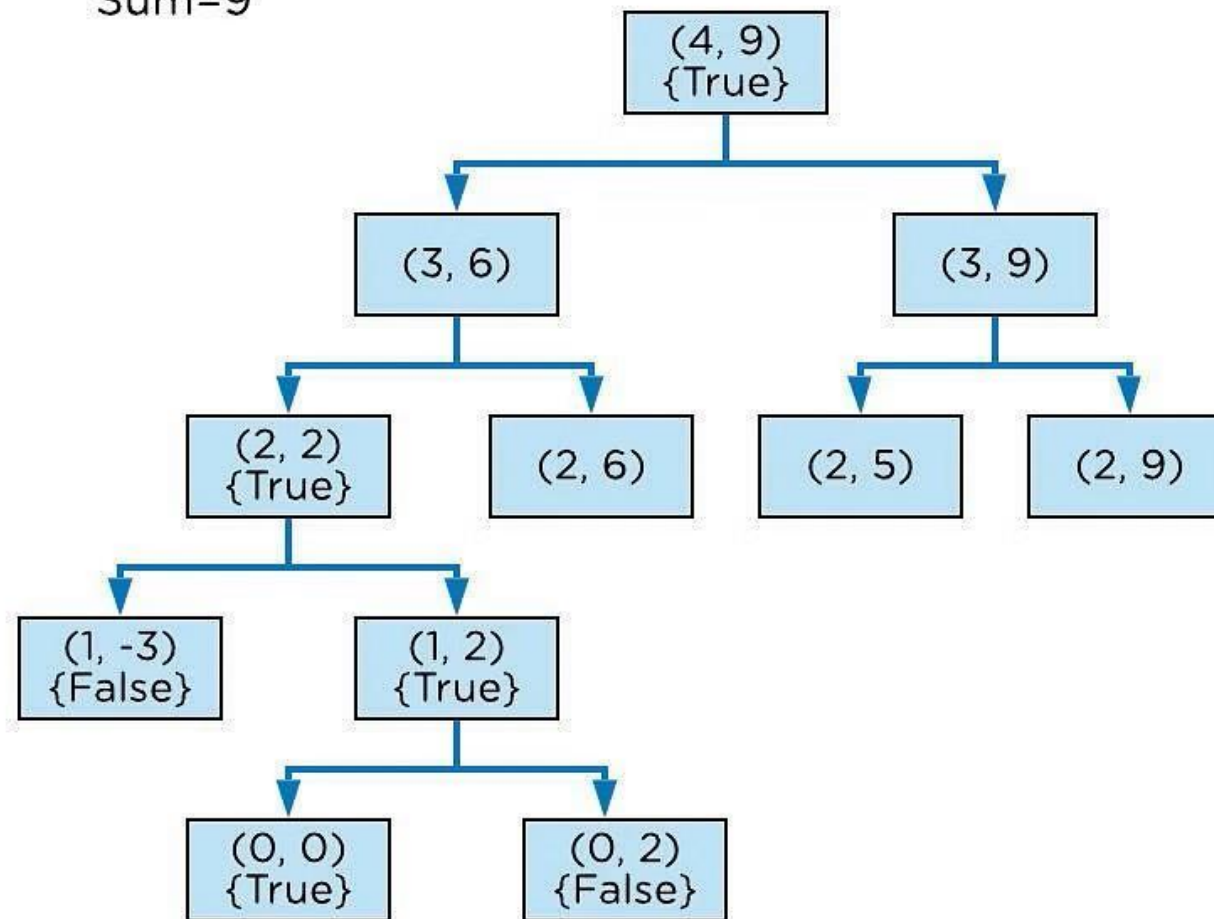
ตัวอย่างปัญหา

- Sum of subsets หรือ subset sum
- 0/1 Knapsack
- Graph coloring
- Job assignment
- Travelling Salesman
- Puzzles
 - Maze runner
 - N-Queens
 - Sudoku

- 
- Decision problems
 - Optimization problems
 - Enumeration problems

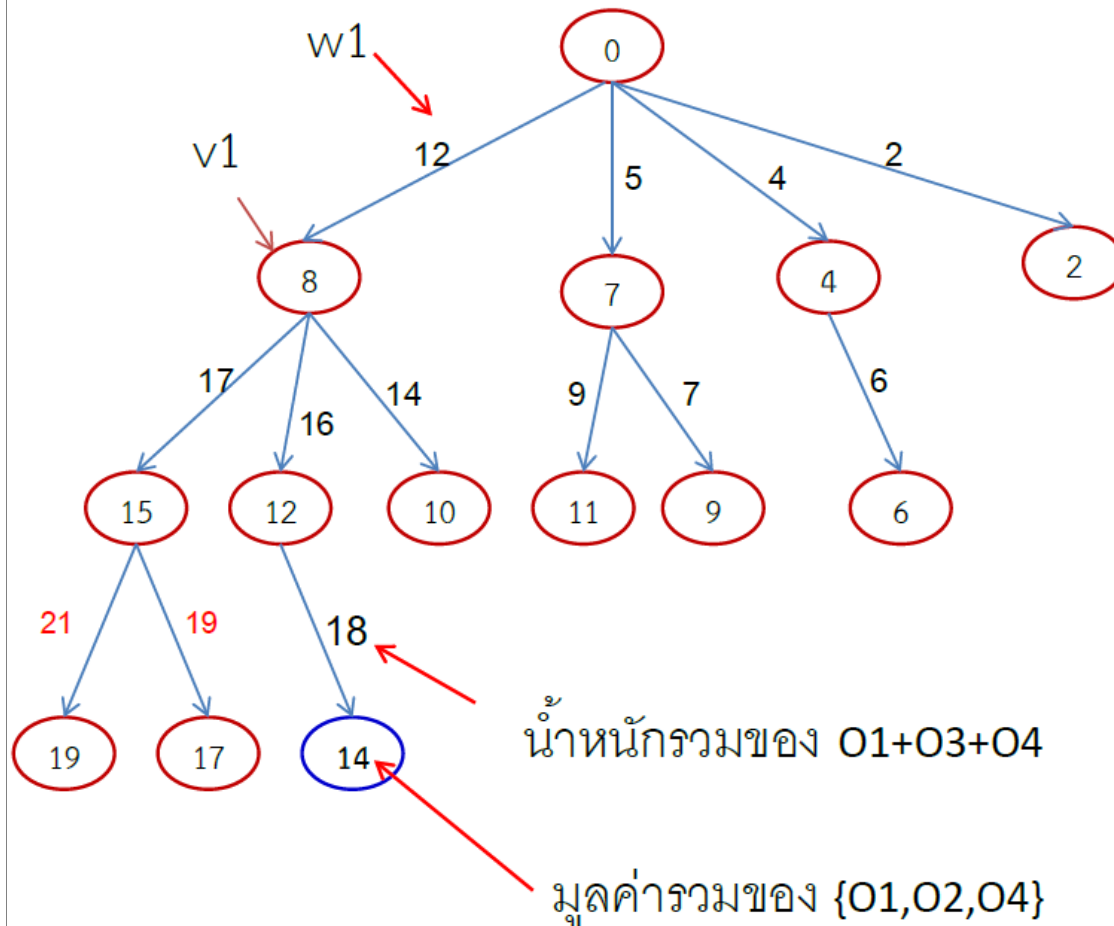
Sum of subsets

Set[]={3, 4, 5, 2}
Sum=9



0/1 Knapsack

Exhaustive Search for knapsack problem

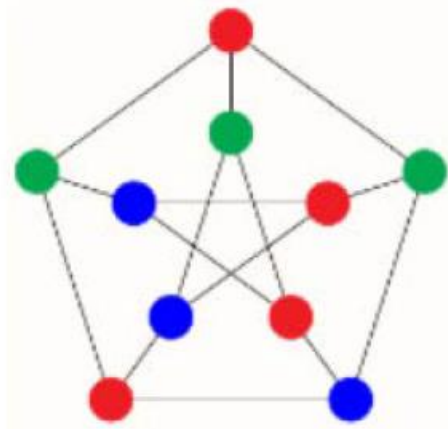
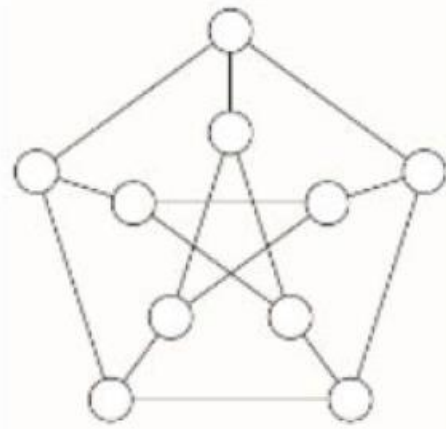


Object	w_i	v_i
O1	12	8
O2	5	7
O3	4	4
O4	2	2
Σ	23	21

$$W = 18$$

Graph coloring

กำหนดให้กราฟ G โหนด หากต้องการระบายสีโหนดบนกราฟ โดยมีเงื่อนไขว่า โหนดที่มีเอดจ์ร่วมกันจะต้องระบายสีแตกต่างกันจะมีวิธีที่เป็นไปได้ (มีหลายคำตอบ) หรืออาจถามว่าจำนวนสีน้อยที่สุด (มีคำตอบเดียว) ที่ใช้ระบายสีบนพื้นที่นี้ตามเงื่อนไข

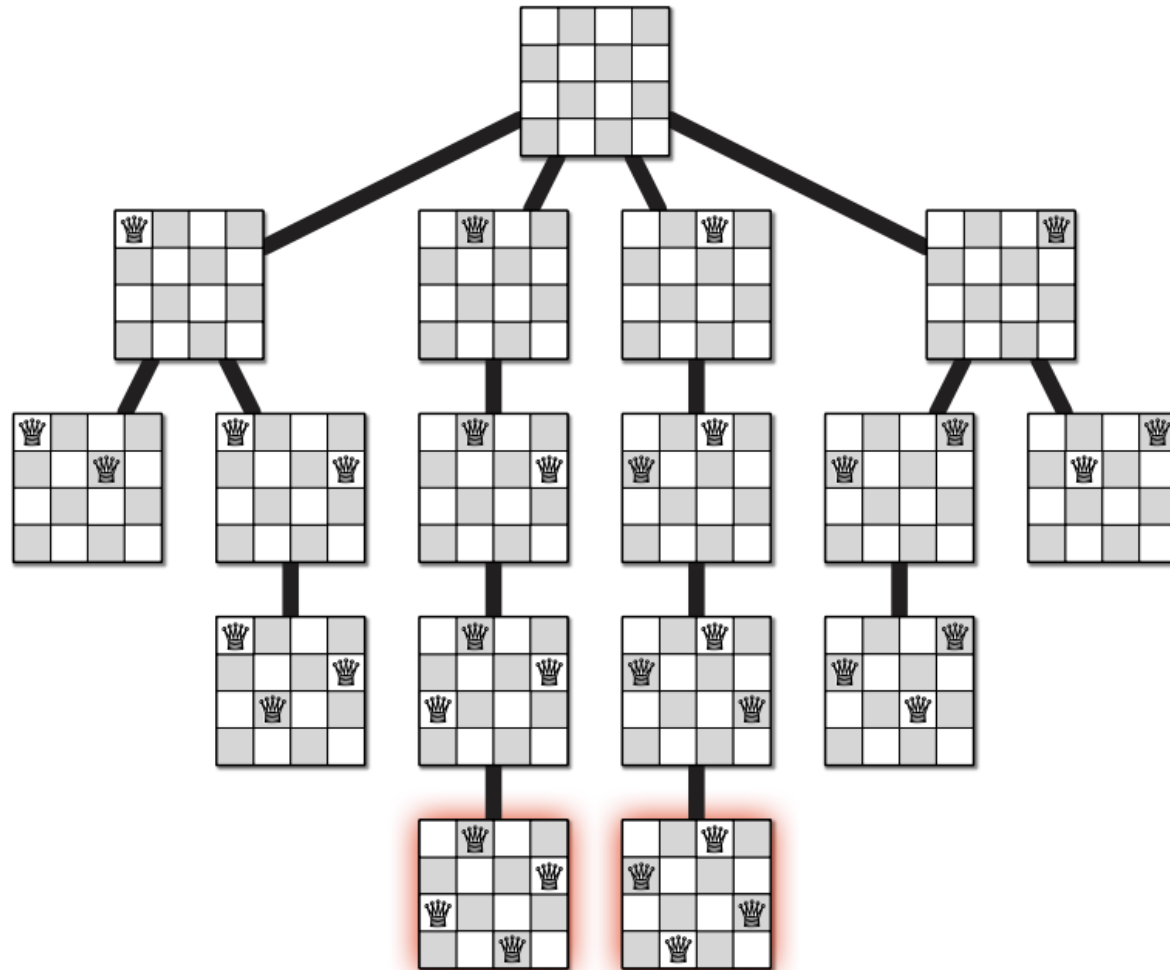


Graph coloring



Map coloring

N-Queens



```

1  #include <stdio.h>
2  #define MAX_N 100
3  int n, x[MAX_N];
4
5  int check(int k, int i) {
6
7  }
```

```

24 int main() {
25     n = 4;
26     n_queens(0);
27     return 0;
28 }
```

```

8 void n_queens(int k) {
9     int i;
10    if (k == n) {
11        for (i = 0; i < n; i++) {
12            printf("%d ", x[i]);
13        }
14        printf("\n");
15    } else {
16        for (i = 0; i < n; i++) {
17            if (check(k, i))
18            { x[k] = i;
19              n_queens(k + 1);
20            }
21        }
22    }
23 }
```

ฟังก์ชัน **check(k, i)** จะเช็คการชนกันของควีนตัวที่ **k** ในแถวที่ **i** กับ ควีนที่ถูกวางไว้ก่อนหน้านี้ โดยจะ **return 0** หากเกิดการชนกัน แต่ถ้าไม่ชนก็จะ **return 1**

```
8 int check(int k, int i) {  
9     for (int j = 0; j < k; j++) {  
10        if ( x[j] == i || abs(x[j] - i) == abs(j - k)) {  
11            return 0;  
12        }  
13    }  
14    return 1;  
15 }
```

